

Ejercicio 1:

En el siguiente código identificar:

- las variables y su tipo
- los operadores
- las funciones y sus parámetros
- las estructuras de control
- cuál es la salida por pantalla

```
<?php
function doble ($i) {
return $i*2:
}
$a = TRUE;
$b = "xyz";
$c = 'xyz';
$d = 12;
echo gettype($a);
echo gettype($b);
echo gettype($c);
echo gettype($d);
if (is_int($d)) {
}
$d += 4;
if (is_string($a)) {
echo "Cadena: $a";
}
$d = $a? ++$d: $d*3;
$f = doble ($d++);
$g = $f += 10;
echo $a, $b, $c, $d, $f, $g;
?>
```

- **Variables y su tipo:**

Variable	Valor	Tipo
\$i	parámetro	integer
\$a	TRUE	boolean
\$b	"xyz"	string
\$c	'xyz'	string
\$d	12	integer
\$f	(es un resultado de doble (\$d++);)	integer

Variable	Valor	Tipo
\$g	(es un resultado de \$f += 10;)	integer

- **Operadores:**

* → **multiplicación** (\$i*2, \$d*3)

+= → **asignación compuesta** (\$d += 4, \$f += 10)

++ → **incremento pre** (++\$d) **y post** (\$d++)

? : → **ternario** (\$a ? ++\$d : \$d*3)

- **Funciones y sus parámetros:**

Función	Parámetro	Descripción
doble(\$i)	integer	retorna \$i * 2
gettype(\$var)	cualquier tipo	retorna el tipo como string
is_int(\$d)	integer	retorna true si es entero
is_string(\$a)	boolean	retorna true si es string

- **Estructuras de control:**

if (is_int(\$d)) → condicional

if (is_string(\$a)) → condicional

\$a ? ++\$d : \$d*3 → operador ternario

- **Salida por pantalla:**

```
<?php
function doble ($i) {
return $i*2:
}
$a = TRUE;
$b = "xyz";
$c = 'xyz';
$d = 12;
echo gettype($a);           → "boolean"
echo gettype($b);           → "string"
echo gettype($c);           → "string"
echo gettype($d);           → "integer"
if (is_int($d)) {
```

\$d += 4;	($d = 12 + 4 = 16$)
}	
if (is_string(\$a)) {	
echo "Cadena: \$a";	→ no entra a este if, por lo tanto no imprime nada
}	
\$d = \$a? ++\$d: \$d*3;	(\$a es true, por lo tanto se ejecuta ++\$d -> \$d = 17)
\$f = doble (\$d++);	($f = 17 * 2 = 34$; Luego $d = 17 + 1 = 18$)
\$g = \$f += 10;	($f = 34 + 10 = 44$; $g = 44$)
echo \$a, \$b, \$c, \$d, \$f, \$g;	\$a → imprime "1"
?>	\$b → "xyz"
	\$c → "xyz"
	\$d → 18
	\$f → 44
	\$g → 44

Respuesta final:

booleanstringstringinteger1xyzxyz184444

Ejercicio 2: Indicar si los siguientes códigos son equivalentes.

a)

```
<?php
$i = 1;
while ($i <= 10) {
    print $i++;
}
?>
```

```
<?php
$i = 1;
while ($i <= 10):
    print $i;
    $i++;
endwhile;
?>
```

```
<?php
$i = 0;
do {
    print ++$i;
} while ($i < 10);
?>
```

Iteración	Código 1	Código 2	Código 3
1	Entra \$i = 1 → Imprime 1	Entra \$i = 1 → Imprime 1	Entra \$i = 0 → Imprime 1
2	Entra \$i = 2 → Imprime 2	Entra \$i = 2 → Imprime 2	Entra \$i = 1 → Imprime 2
3	Entra \$i = 3 → Imprime 3	Entra \$i = 3 → Imprime 3	Entra \$i = 2 → Imprime 3
4	Entra \$i = 4 → Imprime 4	Entra \$i = 4 → Imprime 4	Entra \$i = 3 → Imprime 4
5	Entra \$i = 5 → Imprime 5	Entra \$i = 5 → Imprime 5	Entra \$i = 4 → Imprime 5
6	Entra \$i = 6 → Imprime 6	Entra \$i = 6 → Imprime 6	Entra \$i = 5 → Imprime 6
7	Entra \$i = 7 → Imprime 7	Entra \$i = 7 → Imprime 7	Entra \$i = 6 → Imprime 7
8	Entra \$i = 8 → Imprime 8	Entra \$i = 8 → Imprime 8	Entra \$i = 7 → Imprime 8
9	Entra \$i = 9 → Imprime 9	Entra \$i = 9 → Imprime 9	Entra \$i = 8 → Imprime 9
10	Entra \$i = 10 → Imprime 10	Entra \$i = 10 → Imprime 10	Entra \$i = 9 → Imprime 10

Código 1: Este es el bucle while que usa llaves {}.

- Utiliza el operador de post-incremento. Esto significa que primero imprime el valor actual de \$i, y después de imprimirlo, le suma 1 a la variable.
- Por eso arranca en \$i = 1, imprime el 1, y luego la variable pasa a valer 2 para la siguiente evaluación del while.

Código 2: Este bloque hace exactamente lo mismo que el Código 1, pero cambia dos cosas:

- Sintaxis: En lugar de usar llaves {}, utiliza la sintaxis que abre con dos puntos (:) y cierra con endwhile;.
- En lugar de imprimir y sumar en la misma línea, primero imprime el valor (print \$i;) y en la línea de abajo lo incrementa (\$i++;).

Código 3: El do-while con pre-incremento. La diferencia principal del do-while es que el código se ejecuta al menos una vez antes de evaluar la condición.

- Pre-incremento: Al entrar al bucle, se encuentra con `print ++$i`. Este es el operador de pre-incremento. Significa que primero le suma 1 a la variable (pasando de 0 a 1) y luego imprime ese nuevo valor.
- Condición de corte: La condición es `while ($i < 10)`. En la décima iteración, la variable se incrementa a 10, imprime el 10, y al llegar a la condición `10 < 10` (lo cual es falso), el bucle se detiene.

Conclusión

Sí son equivalentes en salida, los tres imprimen:

12345678910

Pero llegan al mismo resultado por caminos distintos, especialmente el Código 3 que arranca en 0, usa pre-incremento y tiene condición `< 10` en lugar de `<= 10`.

b)

```
<?php
for ($i = 1; $i <= 10; $i++) {
    print $i;
}
?>
```

```
<?php
for ($i = 1; ;$i++) {
    if ($i > 10) {
        break;
    }
    print $i;
}
?>
```

```
<?php
$i = 1;
for (;;) {
    if ($i > 10) {
        break;
    }
    print $i;
    $i++;
}
?>
```

```
<?php
for ($i = 1; $i <= 10; print $i, $i++);
```

?>

Iteración	Valor \$i	Código 1	Código 2	Código 3	Código 4
1	\$i = 1	1 <= 10 (Es V) → Imprime 1	1 > 10 (Es F) → Imprime 1	1 > 10 (Es F) → Imprime 1	1 <= 10 (Es V) → Imprime 1
2	\$i = 2	2 <= 10 (Es V) → Imprime 2	2 > 10 (Es F) → Imprime 2	2 > 10 (Es F) → Imprime 2	2 <= 10 (Es V) → Imprime 2
3	\$i = 3	3 <= 10 (Es V) → Imprime 3	3 > 10 (Es F) → Imprime 3	3 > 10 (Es F) → Imprime 3	3 <= 10 (Es V) → Imprime 3
4	\$i = 4	4 <= 10 (Es V) → Imprime 4	4 > 10 (Es F) → Imprime 4	4 > 10 (Es F) → Imprime 4	4 <= 10 (Es V) → Imprime 4
5	\$i = 5	5 <= 10 (Es V) → Imprime 5	5 > 10 (Es F) → Imprime 5	5 > 10 (Es F) → Imprime 5	5 <= 10 (Es V) → Imprime 5
6	\$i = 6	6 <= 10 (Es V) → Imprime 6	6 > 10 (Es F) → Imprime 6	6 > 10 (Es F) → Imprime 6	6 <= 10 (Es V) → Imprime 6
7	\$i = 7	7 <= 10 (Es V) → Imprime 7	7 > 10 (Es F) → Imprime 7	7 > 10 (Es F) → Imprime 7	7 <= 10 (Es V) → Imprime 7
8	\$i = 8	8 <= 10 (Es V) → Imprime 8	8 > 10 (Es F) → Imprime 8	8 > 10 (Es F) → Imprime 8	8 <= 10 (Es V) → Imprime 8
9	\$i = 9	9 <= 10 (Es V) → Imprime 9	9 > 10 (Es F) → Imprime 9	9 > 10 (Es F) → Imprime 9	9 <= 10 (Es V) → Imprime 9
10	\$i = 10	10 <= 10 (Es V) → Imprime 10	10 > 10 (Es F) → Imprime 10	10 > 10 (Es F) → Imprime 10	10 <= 10 (Es V) → Imprime 10
(Corte)	\$i = 11	11 <= 10 (Es F) → Fin bucle	11 > 10 (Es V) → Break	11 > 10 (Es V) → Break	11 <= 10 (Es F) → Fin bucle

Código 1:

```
for ($i = 1; $i <= 10; $i++) { print $i; }
```

Inicializa la variable en 1, evalúa que sea menor o igual a 10, y si es verdadero, ejecuta el cuerpo (imprimir). Al final del cuerpo, incrementa \$i++.

Código 2:

```
for ($i = 1; ; $i++)
```

Acá se omitió la segunda parte del for (la condición central está vacía). Por defecto, se asume que una condición vacía es true (verdadera), creando un bucle infinito. Para evitar que se ejecute por siempre, se agregó un if (\$i > 10) { break; } adentro del cuerpo del bucle. El incremento (\$i++) sigue estando en la cabecera.

Código 3:

```
for (;;)
```

Este código omite las tres partes de la cabecera. Es un bucle infinito (equivalente a while(true)). Como el for no gestiona nada, se tiene que hacer todo manualmente:

- Se declara \$i = 1 afuera.
- Se pone el if con el break adentro para frenarlo.
- Se pone el \$i++ al final del bloque para incrementarlo a mano.

Código 4:

```
for ($i = 1; $i <= 10; print $i, $i++) ;
```

El punto y coma al final significa que el cuerpo del bucle está vacío. En la tercera sección de la cabecera (la del incremento) se imprime. PHP permite ejecutar múltiples instrucciones ahí separadas por comas. Así que en cada iteración evalúa \$i <= 10, si es verdadero, ejecuta el bloque vacío, y al "terminar" esa vuelta, dispara la tercera sección: primero imprime \$i, y luego ejecuta \$i++.

Conclusión

Los cuatro códigos son exactamente equivalentes y producen el mismo resultado. Aunque la sintaxis varía y las instrucciones se ubican en lugares distintos.

c)

```
<?php
if ($i == 0) {
    print "i equals 0";
} elseif ($i == 1) {
    print "i equals 1";
} elseif ($i == 2) {
```

```

    print "i equals 2";
}
?>

<?php
switch ($i) {
    case 0:
        print "i equals 0";
        break;
    case 1:
        print "i equals 1";
        break;
    case 2:
        print "i equals 2";
        break;
}
?>

```

Valor de \$i (Escenario)	Código 1 (if-elseif)	Código 2 (switch)
\$i = 0	Evalúa \$i == 0 (V) → Imprime "i equals 0"	Coincide con case 0: → Imprime "i equals 0" (y corta con break)
\$i = 1	Evalúa \$i == 1 (V) → Imprime "i equals 1"	Coincide con case 1: → Imprime "i equals 1" (y corta con break)
\$i = 2	Evalúa \$i == 2 (V) → Imprime "i equals 2"	Coincide con case 2: → Imprime "i equals 2" (y corta con break)
Cualquier otro valor (ej: 3)	Ninguna condición es (V) → No imprime nada	Ningún case coincide → No imprime nada

Código 1:

- Primero pregunta: ¿\$i es igual a 0?. Si es verdadero (true), imprime el texto y se ignora todo el resto del bloque.
- Si es falso (false), salta al siguiente elseif y vuelve a preguntar: "¿\$i es igual a 1?".
- Hace esto paso a paso. Al no tener un bloque else al final, si \$i vale 5 (o cualquier otro número no contemplado), el código simplemente termina sin imprimir nada.

Código 2:

- La variable \$i se evalúa una única vez al principio de la estructura.
- PHP busca directamente el case que coincida con ese valor.
- El break indica que se debe cortar el switch una vez que terminó de ejecutar el código de ese caso.

Conclusión

Sí son equivalentes, producen exactamente la misma salida.

Ejercicio 3: Explicar para qué se utiliza el siguiente código:

```
a)
<html>
<head><title>Documento 1</title></head>
<body>
<?php
    echo "<table width = 90% border = '1' >";
    $row = 5;
    $col = 2;
    for ($r = 1; $r <= $row; $r++) {
        echo "<tr>";
        for ($c = 1; $c <= $col;$c++) {
            echo "<td>&nbsp;</td>\n";
        }
        echo "</tr>\n";
    }
    echo "</table>\n";
?>
</body></html>
```

Explicación línea por línea:

echo "<table width = 90% border = '1' >";

Abre una tabla HTML que ocupa el 90% del ancho y tiene borde visible.

\$row = 5;

Define cuántas filas (5) y columnas (2) va a tener la tabla.

\$col = 2;

for (\$r = 1; \$r <= \$row; \$r++)

Bucle externo: Se repite 5 veces, una por cada fila.

echo "<tr>";

Abre una fila de la tabla.

for (\$c = 1; \$c <= \$col; \$c++)

Bucle interno: Se repite 2 veces, una por cada columna.

echo "<td> </td>\n";

Genera una celda vacía (-> espacio en blanco celda no colapse).

echo "</tr>\n";

Cierra la fila, una vez que se generaron las 2 celdas.

echo "</table>\n";

Cierra la tabla cuando terminaron las 5 filas.

En **conclusión**, el código se utiliza para generar una tabla HTML de 5 filas × 2 columnas, con celdas vacías y borde visible.

b)

```
<html>
<head><title>Documento 2</title></head>
<body>
<?php
if (!isset($_POST['submit'])) {
?>
    <form action="<?php echo $_SERVER['PHP_SELF']; ?>" method="post">
    Edad: <input name="age" size="2">
    <input type="submit" name="submit" value="Ir">
    </form>
<?php
    }
else {
    $age = $_POST['age'];
    if ($age >= 21) {
        echo 'Mayor de edad';
    }
    else {
        echo 'Menor de edad';
    }
}
?>
</body></html>
```

Explicación línea por línea:

if (!isset(\$_POST['submit']))

Pregunta si fue enviado el formulario:

- `isset()` devuelve true si la variable existe, luego `!isset()` lo niega, por lo tanto, entra si todavía no “tocaron” el botón de envío del form.

<form action="<?php echo \$_SERVER['PHP_SELF']; ?>" method="post">

Muestra el formulario. `$_SERVER['PHP_SELF']` devuelve la ruta del mismo archivo, o sea el formulario se envía a sí mismo.

Edad: `<input name="age" size="2">`

Campo de texto donde el usuario escribe su edad. Tamaño visual de 2.

`<input type="submit" name="submit" value="Ir">`

Botón que cuando se presiona, genera `$_POST['submit']`, lo que activa el `isset()`.

```
else {  
    $age = $_POST['age'];
```

Si el formulario sí fue enviado, extrae la edad ingresada por el usuario desde `$_POST`.

```
    if ($age >= 21) {  
        echo 'Mayor de edad';  
    }  
    else {  
        echo 'Menor de edad';  
    }
```

Evalúa la edad y muestra el mensaje correspondiente.

En **conclusión**, en la primera carga muestra el formulario, y cuando se envía, procesa los datos y muestra si la persona es mayor o menor de edad.

Ejercicio 4:

Si el archivo `datos.php` contiene el código que sigue:

```
<?php  
$color = 'blanco';  
$flor = 'clavel';  
?>
```

Indicar las salidas que produce el siguiente código. Justificar.

```
<?php  
echo "El $flor $color \n";  
include 'datos.php';  
echo " El $flor $color";  
?>
```

`echo "El $flor $color \n";`

`$flor` y `$color` todavía no existen porque el `include` no se ejecutó aún. Por lo tanto ambas variables son tratadas como indefinidas como `string` y se lanzará un `Warning`. Por lo que la salida será: El (con espacios vacíos donde

deberían ir las variables, y un salto de línea)

```
include 'datos.php';
```

Recién acá se definen \$color = 'blanco' y \$flor = 'clavel'.

```
echo " El $flor $color";
```

Ahora sí existen las variables. Por lo tanto la salida es: El clavel blanco

Salida completa:

El

El clavel blanco

Ejercicio 5: Analizar el siguiente ejemplo: Contador de visitas a una página web

Se implementa un contador de visitas a una página web, compuesto por tres archivos: visitas.php, contador.php y contador.dat.

visitas.php

Es la página visible para el usuario. incluye a contador.php:

```
<? include("contador.php")?>
```

Al acceder a visitas.php desde el navegador, se ejecuta automáticamente el código de contador.php, por eso ambos archivos mostrarán el mismo resultado y comportamiento (desde XAMPP).

contador.php

```
$archivo = "contador.dat";
```

Define el nombre del archivo de texto donde se registra el número de visitas.

```
$abrir = fopen($archivo, "r");  
$cont = fread($abrir, filesize($archivo));  
fclose($abrir);
```

Abre contador.dat en modo lectura ("r"), lee todo su contenido (el número actual de visitas) y cierra el archivo.

```
$cont = $cont + 1;  
$abrir = fopen($archivo, "w");  
$guardar = fwrite($abrir, $cont);  
fclose($abrir);
```

Suma 1 al número leído, abre nuevamente contador.dat en modo escritura ("w") — sobrescribiendo el contenido anterior — guarda el nuevo número y cierra el archivo.

```
echo "<font face='arial' size='3'>Cantidad de visitas:". $cont. "</font>";
```

Muestra en pantalla el número actualizado de visitas.

contador.dat

Es un archivo de texto plano que únicamente almacena el número actual de visitas. Se inicializa con el valor 0. No contiene lógica PHP, por lo tanto acceder o recargar este archivo directamente desde el navegador no modifica su contenido.

Comportamiento en la práctica:

Al ejecutar el proyecto en XAMPP y acceder a visitas.php desde el navegador, se observa un texto que indica la cantidad de visitas. Cada vez que se recarga la página, el número aumenta en 1. Lo mismo ocurre al acceder directamente a contador.php. Al abrir contador.dat, se puede verificar que el número almacenado coincide con el mostrado en pantalla.

PHP: arrays, funciones

Ejercicio 1: Indicar si los siguientes códigos son equivalentes.

```
<?php
$a = array( 'color' => 'rojo',
           'sabor' => 'dulce',
           'forma' => 'redonda',
           'nombre' => 'manzana',
           4
         );
?>
```

```
<?php
$a['color'] = 'rojo';
$a['sabor'] = 'dulce';
$a['forma'] = 'redonda';
$a['nombre'] = 'manzana';
$a[] = 4;
?>
```

Código 1:

Este código define y llena el arreglo de una sola vez usando el constructor array().

- `$a = array(...` : Crea un nuevo arreglo vacío y se lo asigna a la variable `$a`. Si `$a` ya existía, su contenido anterior se borra.
- `'color' => 'rojo', (hasta 'nombre' => 'manzana',)`: Asigna valores específicos a claves de texto.
- `4`: Este elemento no tiene una clave definida como los anteriores. Cuando se inserta un valor sin especificar su clave, se le asigna automáticamente el siguiente número

entero disponible empezando desde el 0. Como no había ningún número antes, a este valor 4 se le asigna la clave numérica 0.

Código 2:

Este enfoque va construyendo el arreglo paso a paso, agregando una clave a la vez.

- `$a['color'] = 'rojo';` (hasta `$a['nombre'] = 'manzana';`): PHP verifica si la variable `$a` existe. Si no existe, la crea como un arreglo en ese mismo momento. Luego, va agregando cada clave de texto con su respectivo valor.
- `$a[] = 4;`: Los corchetes vacíos `[]` son el operador de append. Al igual que en el código anterior, como no hay claves numéricas previas, le asigna la clave 0.

La estructura interna del arreglo en memoria sería idéntica en ambos casos:

Clave interna (Key)	Valor almacenado (Value)	Tipo de dato
'color'	'rojo'	String
'sabor'	'dulce'	String
'forma'	'redonda'	String
'nombre'	'manzana'	String
0	4	Integer

Conclusión

Sí son equivalentes. Producen exactamente el mismo array. La única diferencia es la forma de escribirlo: el Código 1 crea todo el array de una sola vez, mientras que el Código 2 lo construye incrementalmente, asignando clave por clave.

Ejercicio 2: En cada caso, indicar las salidas correspondientes:

```
a)
<?php
$matriz = array("x" => "bar", 12 => true);
echo $matriz["x"];
echo $matriz[12];
?>
```

Análisis del código:

```
$matriz = array("x" => "bar", 12 => true);
```

Crea un array asociativo con dos elementos:

Clave	Valor	Tipo
"x"	"bar"	string
12	true	boolean

echo \$matriz["x"];

Accede por clave "x" → imprime "bar"

echo \$matriz[12];

Accede por clave 12 → el valor es true (boolean).

Cuando se hace echo con un booleano:

- true → imprime "1"
- false → imprime ""

Salida final:

bar1

b)

```
<?php
```

```
$matriz = array("unamatriz" => array(6 => 5, 13 => 9, "a" => 42));
```

```
echo $matriz["unamatriz"][6];
```

```
echo $matriz["unamatriz"][13];
```

```
echo $matriz["unamatriz"]["a"];
```

```
?>
```

Análisis del código:

\$matriz = array("unamatriz" => array(6 => 5, 13 => 9, "a" => 42));

Es un array que contiene otro array adentro:

Clave exterior	Clave interior	Valor
"unamatriz"	6	5
"unamatriz"	13	9
"unamatriz"	"a"	42

echo \$matriz["unamatriz"][6];

Accede al array exterior por "unamatriz", luego al interior por 6 → imprime 5

echo \$matriz["unamatriz"][13];

Accede al array exterior por "unamatriz", luego al interior por 13 → imprime 9

```
echo $matriz["unamatriz"]["a"];
```

Accede al array exterior por "unamatriz", luego al interior por "a" → imprime 42

Salida final

5942

c)

```
<?php
```

```
$matriz = array(5 => 1, 12 => 2);
```

```
$matriz[] = 56;
```

```
$matriz["x"] = 42;
```

```
unset($matriz[5]);
```

```
unset($matriz);
```

```
?>
```

Análisis del código:

```
$matriz = array(5 => 1, 12 => 2);
```

Clave	Valor
5	1
12	2

```
$matriz[] = 56;
```

Sin clave explícita: Se toma el índice más alto y se le suma 1 → $12+1 = 13$

Clave	Valor
5	1
12	2
13	56

```
$matriz["x"] = 42;
```

Agrega elemento con clave string "x"

Clave	Valor
5	1
12	2

Clave	Valor
13	56
"x"	42

unset(\$matriz[5]);

Elimina el elemento con clave 5

Clave	Valor
12	2
13	56
"x"	42

unset(\$matriz);

Destruye el array completo — la variable deja de existir.

Salida final

(ninguna)

Ejercicio 3: En cada caso, indicar las salidas correspondientes

a)

```
<?php
```

```
$fun = getdate();
```

```
echo "Has entrado en esta pagina a las $fun[hours] horas,  
con $fun[minutes] minutos y $fun[seconds] segundos,  
del $fun[mday]/$fun[mon]/$fun[year]";
```

```
?>
```

Análisis del código:

\$fun = getdate();

getdate() sin parámetros devuelve un array asociativo con la fecha y hora local actual del servidor. Almacena todo en \$fun.

**echo "Has entrado en esta pagina a las \$fun[hours] horas,
con \$fun[minutes] minutos y \$fun[seconds] segundos,
del \$fun[mday]/\$fun[mon]/\$fun[year]";**

Accede a las claves del array dentro del string:

Clave	Contiene
\$fun[hours]	hora (0-23)
\$fun[minutes]	minutos (0-59)
\$fun[seconds]	segundos (0-59)
\$fun[mday]	día del mes (1-31)
\$fun[mon]	mes (1-12)
\$fun[year]	año (4 dígitos)

Salida final:

Ejemplo:

Has entrado en esta pagina a las 14 horas,
con 35 minutos y 22 segundos,
del 7/9/2026

b)

```
<?php
```

```
function sumar($sumando1, $sumando2){
    $suma = $sumando1 + $sumando2;
    echo $sumando1."+".$sumando2."=".$suma;
}
sumar(5, 6);
?>
```

Análisis del código:

function sumar(\$sumando1, \$sumando2)

Define una función llamada sumar que recibe dos parámetros por valor.

\$suma = \$sumando1 + \$sumando2;

Suma los dos parámetros y guarda el resultado en \$suma.

echo \$sumando1."+".\$sumando2."=".\$suma;

Concatena con . y muestra la operación completa en formato legible.

sumar(5, 6);

Llama a la función con \$sumando1 = 5 y \$sumando2 = 6.

Salida final:

5+6=11

Ejercicio 4: Analizar la siguiente función, y escribir un script para probar su funcionamiento

```
function comprobar_nombre_usuario($nombre_usuario){
    if (strlen($nombre_usuario)<3 || strlen($nombre_usuario)>20){
        echo $nombre_usuario . " no es válido<br>";
        return false;
    }
    $permitidos =
"abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789-_" ;
    for ($i=0; $i<strlen($nombre_usuario); $i++){
        if (strpos($permitidos, substr($nombre_usuario,$i,1))===false){
            echo $nombre_usuario . " no es válido<br>";
            return false;
        }
    }
    echo $nombre_usuario . " es válido<br>";
    return true;
}
```

Análisis del código:

function comprobar_nombre_usuario(\$nombre_usuario)

Define una función que recibe un string (con el nombre de usuario a validar) y devuelve true o false según si es válido o no.

if (strlen(\$nombre_usuario)<3 || strlen(\$nombre_usuario)>20)

Usa strlen() para verificar que el nombre tenga entre 3 y 20 caracteres. Si es más corto o más largo:

**echo \$nombre_usuario . " no es válido
";**

return false;

Informa que no es válido y termina la función inmediatamente con false. No sigue evaluando.

\$permitidos =

"abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789-_" ;

Define un string con todos los caracteres válidos: letras minúsculas, mayúsculas, números, guión medio y guión bajo. Cualquier otro carácter (como @, #, espacios, etc.) no está permitido.

for (\$i=0; \$i<strlen(\$nombre_usuario); \$i++)

Recorre el nombre carácter por carácter, desde el primero hasta el último.

```
substr($nombre_usuario, $i, 1)
```

Extrae el carácter en la posición \$i (longitud 1).

```
strpos($permitidos, substr($nombre_usuario,$i,1)) === false
```

Busca ese carácter dentro del string \$permitidos.

Usa === → strpos() puede devolver 0 (posición 0) y usando == se confundiría con false.

Si el carácter no se encuentra:

```
echo $nombre_usuario . " no es válido<br>";
```

```
return false;
```

Informa que no es válido y termina inmediatamente.

```
echo $nombre_usuario . " es válido<br>";
```

```
return true;
```

Solo llega acá si el nombre tiene longitud correcta y todos sus caracteres son permitidos.

La función realiza una validación de nombre de usuario en dos etapas: primero controla la longitud, luego verifica carácter por carácter que solo contenga letras, números, guión medio y guión bajo.

Script de prueba:

```
// Casos de prueba
```

```
echo "<h3>Pruebas de validación:</h3>";
```

```
// Caso 1: nombre válido normal
```

```
comprobar_nombre_usuario("juan23");
```

```
// Caso 2: nombre válido con guiones
```

```
comprobar_nombre_usuario("juan_perez-23");
```

```
// Caso 3: muy corto (menos de 3 caracteres)
```

```
comprobar_nombre_usuario("ab");
```

```
// Caso 4: muy largo (más de 20 caracteres)
```

```
comprobar_nombre_usuario("esteNombreEsDemasiadoLargo123");
```

```
// Caso 5: caracteres inválidos
```

```
comprobar_nombre_usuario("juan@#!");
```

```
// Caso 6: con espacio
```

```
comprobar_nombre_usuario("juan perez");
```

Salida:

Pruebas de validación:

juan23 es válido

juan_perez-23 es válido

ab no es válido

esteNombreEsDemasiadoLargo123 no es válido

juan@#! no es válido

juan perez no es válido

Análisis de cada caso

Caso	Nombre	Resultado	Motivo
1	"juan23"	válido	Longitud correcta, solo letras y números
2	"juan_perez-23"	válido	Longitud correcta, guiones permitidos
3	"ab"	no válido	Menos de 3 caracteres
4	"esteNombreEsDemasiadoLargo123"	no válido	Más de 20 caracteres
5	"juan@#!"	no válido	Contiene @, #, ! — no permitidos
6	"juan perez"	no válido	El espacio no está entre los caracteres permitidos

Los resultados confirman el correcto funcionamiento de la función. Las dos validaciones operan como se esperaba: primero se controla la longitud del nombre y luego se verifica carácter por carácter que solo contenga los caracteres permitidos. Cualquier nombre que no cumpla alguna de las dos condiciones es rechazado inmediatamente.